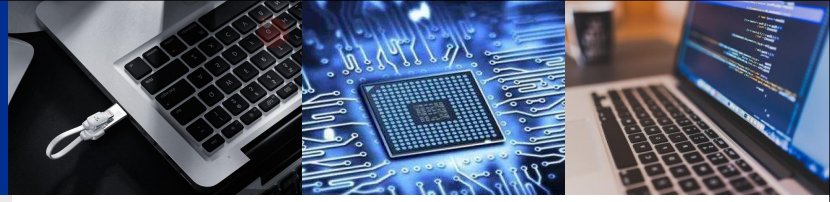




4 汇编语言程序设计

- 汇编语言程序基本格式
- 汇编语言中的数据
- 运算符与表达式
- 伪指令
- 系统功能调用
- 宏指令
- 汇编语言程序设计举例
- 汇编语言程序上机过程
- 调试程序DEBUG的使用



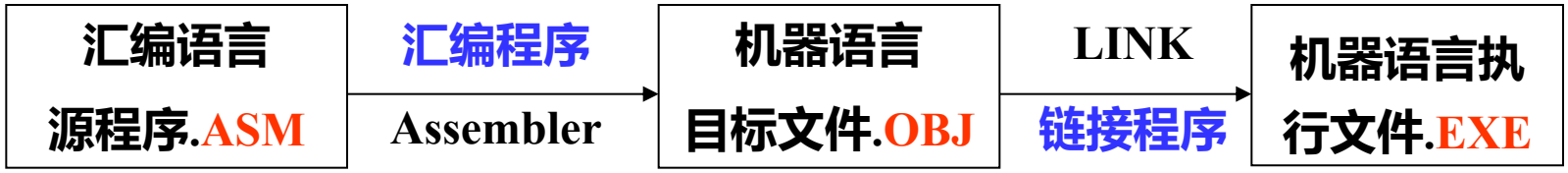
4.1汇编语言程序基本格式

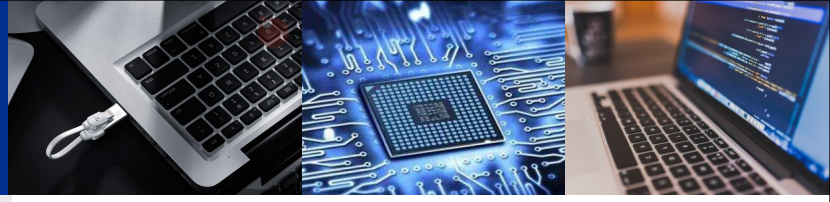
4.1.1汇编语言源程序和汇编程序

汇编程序以汇编语言源程序文件作为输入，并由它产生两种输出文件：目标程序文件和源程序列表文件。

- 目标程序文件经**链接定位**后由计算机执行；
- 源程序列表文件将列出源程序、目标程序的机器语言代码及符号表。

汇编语言源程序的处理过程如下：





4.1汇编语言程序基本格式

4.1.2汇编语言的特点

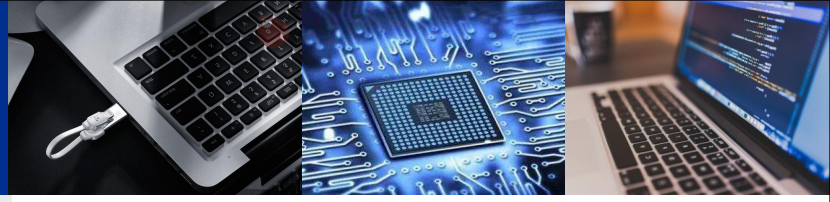
① 执行速度快。

② 程序短小。

③ 可以直接控制硬件。

④ 可以方便地编译。

⑤ 辅助计算机工作者掌握计算机体系结构。



4.1.3一般汇编语言程序的结构形式

```
stack   SEGMENT  PARA  'stack'
        DB   100  DUP('stack')
stack   ENDS
```

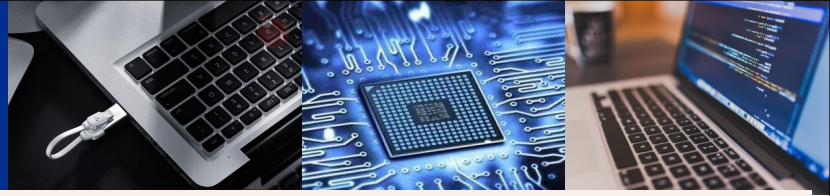
堆栈段

```
data    SEGMENT
        <数据、变量在此定义>
data    ENDS
```

数据段

```
code    SEGMENT
        ASSUME  CS:code, DS:data, ES:data, SS:stack
start:  MOV    AX, data
        MOV    DS, AX
        MOV    ES, AX
        <此处加入你自己的程序段>          真指令
        MOV    AL, 4CH
        INT    21H
code    ENDS
        END    start
```

代码段



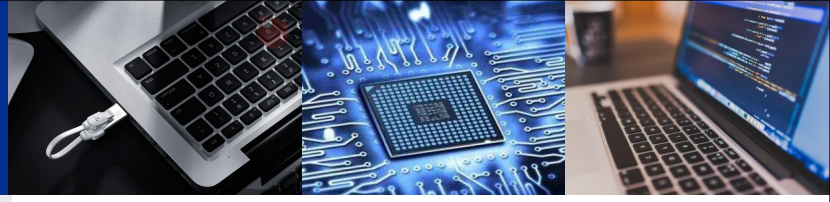
Masm for Windows 集成实验环境共享版 2015

文件(F) 编辑(E) 运行(R) 工具(O) 帮助(H)

新建 打开 保存 剪切 复制 粘贴 查找 注释 取注 撤销 恢复 运行 列表 调试 教程 日记 注册 网站 帮助

- 我的程序
  - 1.asm
- 软件应用问题解答
- 简单的入门程序实例
- 数据传送指令(免费)
- 算术运算指令
- 逻辑运算指令
- 串操作指令
- 程序转移指令
- 汇编伪指令
- DOS汇编源程序
- WINDOWS汇编实例源程序
- 汇编语言错误信息表
- DOS 功能调用表(免费)
- BIOS 功能调用表(免费)
- DEBUG 命令(免费)
- ASCII码字符表(免费)

```
01 DATAS SEGMENT
02     ;此处输入数据段代码
03 DATAS ENDS
04
05 STACKS SEGMENT
06     ;此处输入堆栈段代码
07 STACKS ENDS
08
09 CODES SEGMENT
10     ASSUME CS:CODES,DS:DATAS,SS:STACKS
11 START:
12     MOV AX,DATAS
13     MOV DS,AX
14     ;此处输入代码段代码
15     MOV AH,4CH
16     INT 21H
17 CODES ENDS
18     END START
```



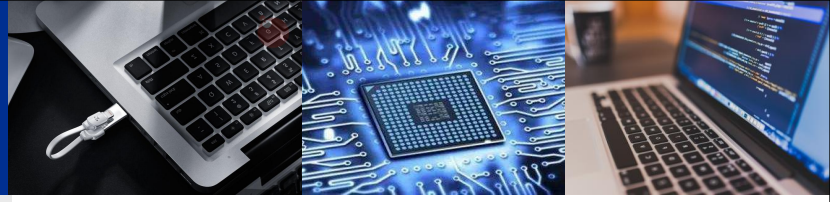
4.1汇编语言程序基本格式

每一段的语句体由语句序列组成，8086汇编语言语句分为如下三类。

① **指令语句**：8086指令系统的指令形式，与机器指令一一对应。

② **伪指令语句**：又称管理语句。在汇编语言源程序的汇编过程中起作用，它是对汇编程序的命令语句，一般没有相应的目标代码。

③ **宏指令语句**：是宏汇编程序能识别的、预先定义的指令代码序列。一旦定义以后，宏指令就像一条指令一样，可以在源程序中被引用，其效果等同于引入一段代码序列。



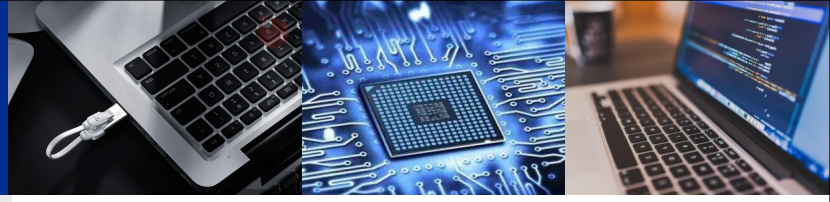
4.1汇编语言程序基本格式

一般情况下，汇编语言的**指令性语句**可以由以下几部分组成：

[标号:] [前缀] 操作码助记符 [操作数] [；注释]

伪指令语句又称为**指示性语句**，其格式为：

[名字] 伪指令定义符 [操作数] [； 注释]



4.2汇编语言中的数据

4.2.1常量

- **数字常量：**可以是二进制数、八进制数、十进制数或十六进制数表示。

二进制：00011101B

十六进制：0C5AAH

十进制：9856D

八进制：237O

- **字符常量：**是用单引号‘ ’括起来的ASCII字符，其值是该字符的ASCII代码值。

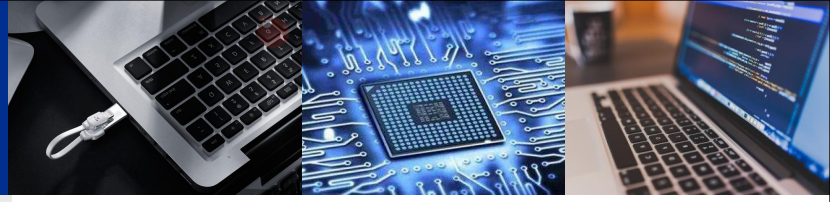
’The 2X means 2 multiple X’

- **符号常量：**是用标识符定义的常量。

例如，采用符号定义语句 **PORTA EQU 80H**，

则指令 **MOV AL，PORTA**

与指令 **MOV AL，80H** 等价。



4.2汇编语言中的数据

4.2.2 变量

1. 数据定义伪指令

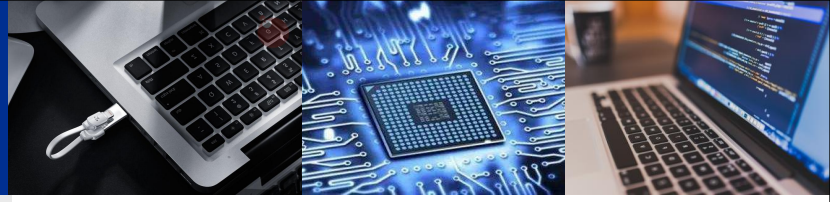
数据定义伪指令用来定义一个变量，并将所需要的数据放入指定的存储单元中，也可以只给变量分配存储单元，而不赋予特定的值。

- 数据定义伪指令的一般格式为：

  [变量名]　数据定义符　操作数 [，操作数…]　[；注释]

  - **常用的数据定义伪指令**

    - **定义字节变量伪指令　DB**
    - **定义字变量伪指令　DW**
    - **定义双字变量伪指令　DD**
    - **定义四字变量伪指令　DQ**
    - **定义十字节变量伪指令　DT**
    - **经过定义的变量有三个属性：数据类型、偏移量、段基址**



4.2汇编语言中的数据

表达式是给变量预置的初值，可以是下述情况之一：

①数值表达式：数值允许用二进制、八进制、十进制、十六进制形式书写。

②？：表示不预置确定的初值。

③字符串表达式：用引号括起来的不超过255个字符或其他ASCII码符号。DB伪指令将按顺序为字符串中每一个字符或符号分配一个字节单元，存放它们的ASCII编码，但DB以外的数据定义伪指令只允许定义最多两个字符的字符串，且按逆序存放在低地址开始的单元。

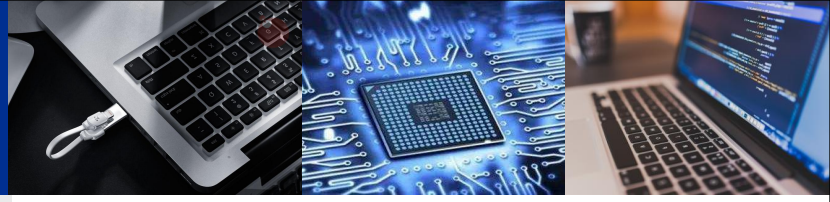
注意：3个及其以上的字符，只能用DB定义。当字符串的长度不超过两个字符时可用DW定义。

```
str1   DW   'abcd'
str2   DD   'abcd'
```



④带DUP操作符的表达式：DUP是定义重复数据操作符，它的使用格式是：

N DUP (Exp)；其中，N为重复次数，EXP为表达式。



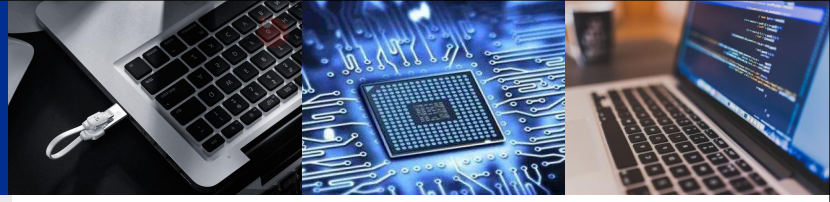
4.2汇编语言中的数据

例如：

```
DATA1   DB   25,25H,10011010B;数值表达式
DATA2   DB   ?,?;?表达式
DATA3   DB   2 DUP(2 DUP(4),15);DUP表达式
DATA4   DB   'AB','CD';字符串表达式
DATA5   DW   ?,?,-32768;字类型
```

DATA1	19H
	25H
	9AH
DATA2	?
	?
DATA3	04H
	04H
	0FH
	04H
	04H
	0FH

DATA4	42H
	41H
	44H
	43H
DATA5	?
	?
	?
	?
	00H
	80H



4.2汇编语言中的数据

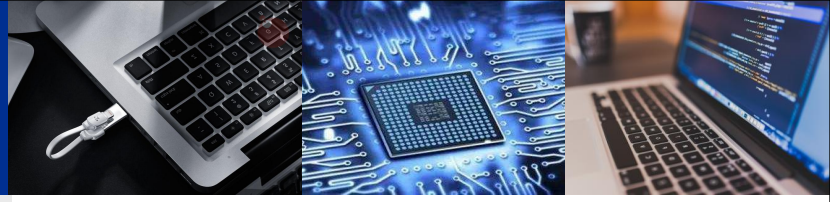
2.变量的属性

变量有三种属性：**段属性、偏移属性、类型属性。**

① **段属性**（SEGMENT）：变量所在的段的起始地址（16位），此值必须在一个段寄存器中。

② **偏移属性**（OFFSET）：该变量与段的起始地址之间相距的字节数。对于16位段，是16位无符号数; 对于32位段，则是32位无符号数。在当前段内给出的变量的偏移值等于当前地址计数器的值，当前地址计数器的值可以用$来表示。

③ **类型属性**（TYPE）：定义该变量的字节数。如BYTE（DB，1B长）、WORD（DW，2B长）、DWORD（DD，4B长）、FWORD（DF，6B长）、QWORD（DQ，8B长）、TBYTE（DT，10B长）。



4.2汇编语言中的数据

3.变量的使用

① 变量名作为存储单元的**直接地址**：变量名用直接寻址时, 变量的类型必须与指令的要求相符合。

例如: AB 已定义字节变量, AW 定义为字变量, 用变量名作直接寻址形式如下:

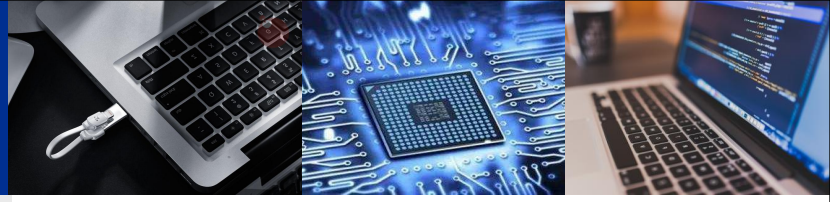
```
MOV   AH,AB
MOV   AX, AW
```

② 用合成运算符**PTR**可以临时改变变量类型。

例如，AB 已定义字节变量, AW 定义为字变量，指令序列如下:

```
MOV   CX, WORD PTR AB
MOV   CL, BYTE PTR AW
```

则临时把AB 变为字类型, AW 变为字节类型, 但段和偏移属性不变。



4.2汇编语言中的数据

③ 变量名作为相对寻址中的偏移量

例如，AB 已定义字节变量, AW 定义为字变量，在如下指令序列中：

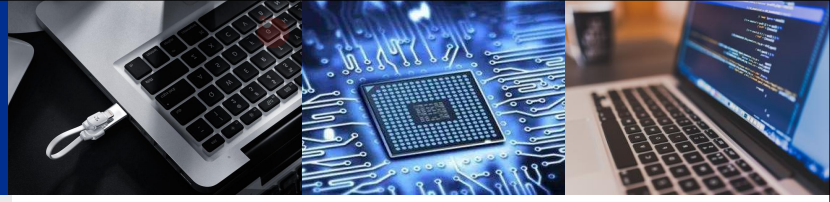
```
MOV  AX, AB[SI]
MOV  AX, AW[ BX][ SI]
```

AB, AW 分别表示它们的偏移量而不是它们所表示的数据，常用于数组或表格操作，AB[SI] 就表示AB 数组中第SI个元素。

④ 变量名仅对应数据区第一个数据项

例如:

```
WORD   DW 20 DUP( ?)
MOV    AX, WORD                ; 第一个元素送AX,
MOV    AX, WORD+ 38            ; 第20 个元素送AX.
```



4.2汇编语言中的数据

4.2.3 标号

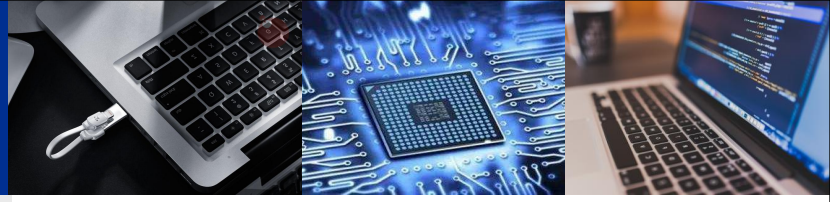
- 在指令性语句中，标号表示本条语句的**符号地址**；
- 标号、名字统称为标识符，都具备3种属性

——**段属性、偏移属性及类型属性**。

① 标号的段属性是定义标号在程序段的段地址。当程序中引用一个标号时，该标号的段值应在CS寄存器中。

② 标号的偏移量属性表示标号所在段的起始地址到定义该标号的地址之间的字节数。偏移量是一个16位无符号数。

③ 标号的类型属性有两种：NEAR和FAR。前一种标号可以在段内被引用，地址指针为2字节；后一种标号可以在其他段被引用，地址指针为4字节。



4.3运算符与表达式

表达式由操作数和运算符组成，如3+2、77 AND 55、NOT 5AH等。

1. 操作数

操作数用于代表一个数据或一个地址。

2. 运算符

运算符包括算术、逻辑、关系、分析、综合五类运算符。

① 算术运算符 包括 +、 -、*、/ 、MOD

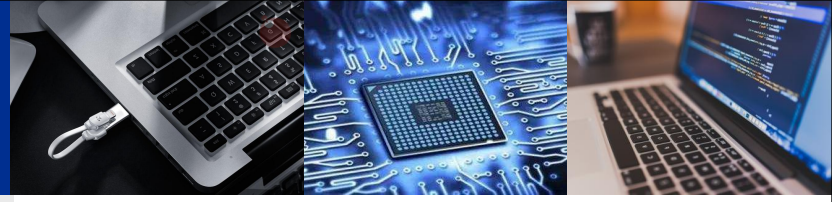
参加运算的内容可以是数据，也可以是地址，但在一个算术运算符的两边不能都是地址。

② 逻辑运算符 包括 AND、OR、NOT、XOR

③ 关系运算符 包括 相等：EQ 大于：GT

不等：NE 小于或等于：LE

小于：LT 大于或等于：GE



4.3运算符与表达式

关系运算结果只有两个——真、假。

如果关系是真，则结果为 0FFH（0FFFFH）；

如果关系是假，则结果为0。

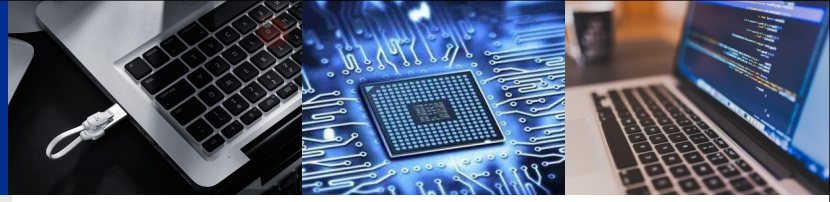
例如

MOV BX, DATA1 LT 0010H ⟺ MOV BX, 0FFFFH

当 DATA1< 0010H 时

MOV BX, DATA1 LT 0010H ⟺ MOV BX, 0

当 DATA1 > 0010H 时



4.3运算符与表达式

④取值运算符

取值运算在汇编语言程序设计中很重要。它可以把它存储器操作数分解成它的组成部分:

包括

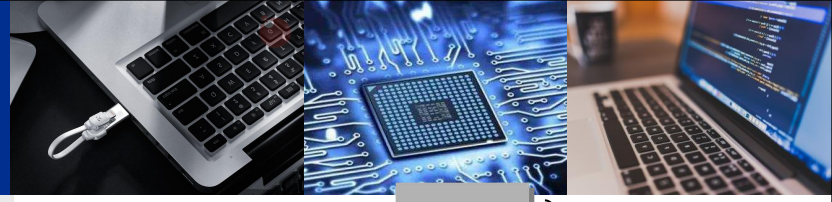
取存储单元偏移：OFFSET

取存储单元段： SEG

取类型：TYPE　TYPE：返回类型值：BYTE-1 WORD-2 DWORD-4 NEAR- -1 FAR- -2

取字节数： SIZE=LENGTH*TYPE

按类型取长度：LENGTH



4.3运算符与表达式

例1:设有数据段中使用

DATA1 DB ?

定义了一个单元，则

MOV AL, DATA1

可将DATA1中的数传送到AL中， AL=05

MOV AX, SEG DATA1

可将DATA1的所在段地址传到AX中，AX=2000

MOV AX, OFFSET DATA1

可将DATA1的偏移地址传入AX中， AX=1500

例2: 设有数据段中使用

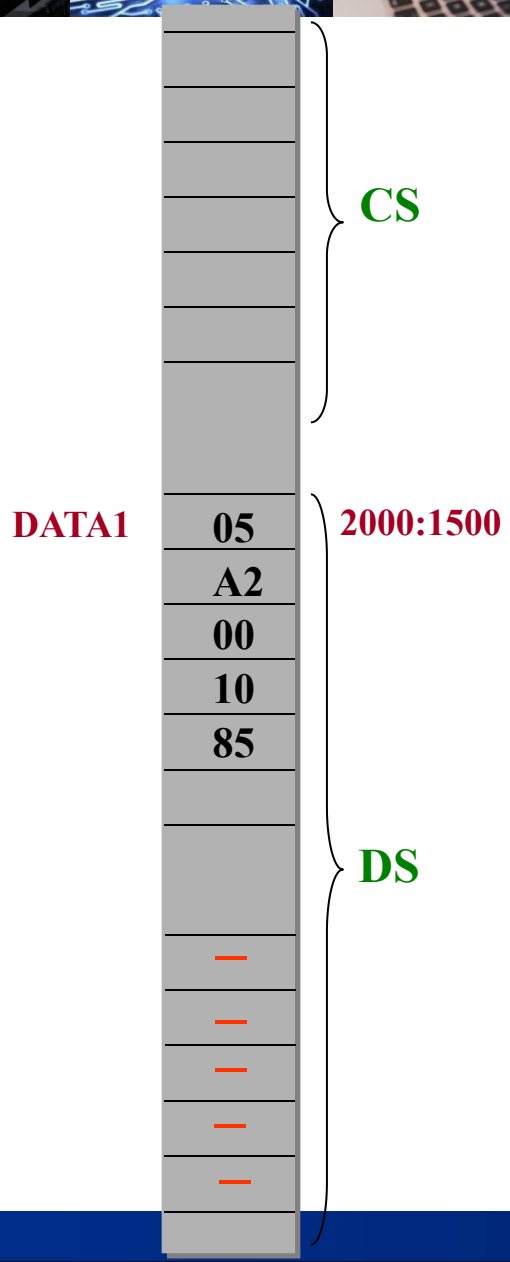
DATA1 DW 100 DUP (?)

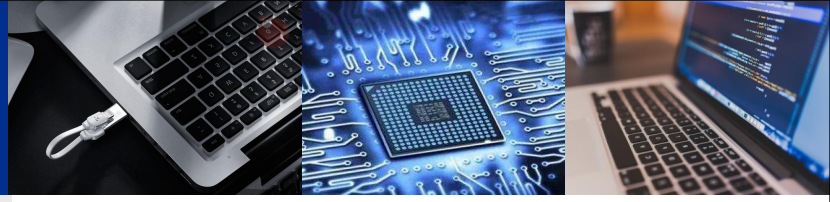
定义了100个字单元，则

MOV AX, TYPE DATA1 AX=2

MOV AX, SIZE DATA1 AX=200

MOV AX, LENGTH DATA1 AX=100





4.3运算符与表达式

⑤ 综合运算符

综合运算符包括两个地址操作符 PTR

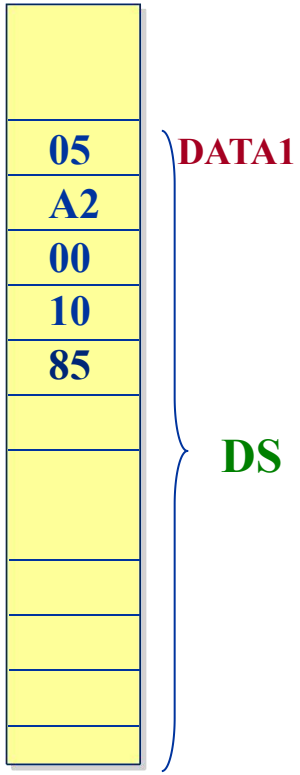
PTR——改变存储器地址操作数的操作类型，但其段地址和偏移地址不变。

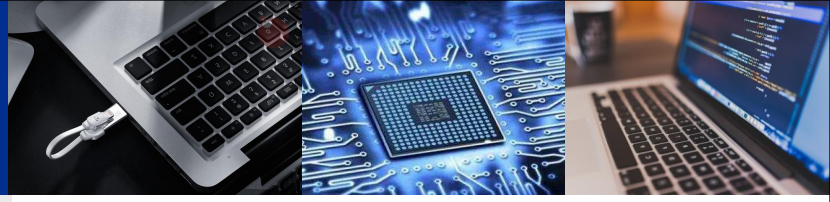
如：

DATA1 DW 100 DUP(?)

MOV AL, DATA1 ×

MOV AL, BYTE PTR DATA1 √





4.3运算符与表达式

在计算具有多个运算符的表达式值时，按以下规则操作：

◎优先级高的先运算，优先级低的后运算。

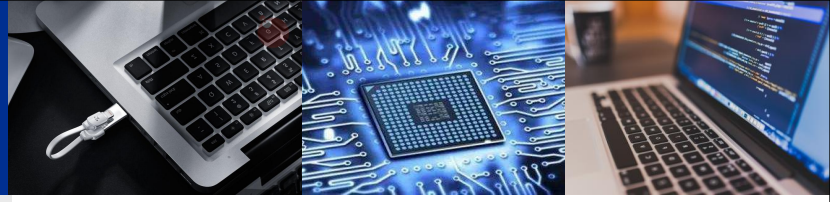
◎当优先级相同时，按表达式中从左到右的顺序进行运算

◎使用括号可以提高表达式运算的优先级。

◎表达式中的运算符的优先级

THIS :把运算符后面指出的类型属性赋给当前的存储单元

优先级		
高	1	括号中的项，即（…）和[…]
	2	LENGTH，SIZE，WIDTH，MASK
	3	PTR，OFFSET，SEG，TYPE，THIS
	4	×，/，MOD
	4	+，-
	6	EQ，NE，L，LE，GT，GE
	7	NOT
	8	AND
低	9	OR，XOR



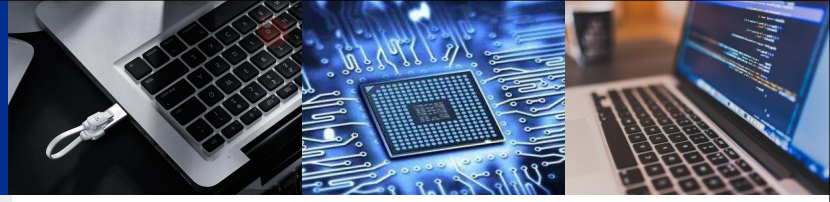
4.4伪指令

在8086/8088汇编语言中伪指令是用于诸如数据定义、存储区分配等功能。所谓伪指令是非机器指令，它是在汇编期间进行操作的。

1. 符号定义伪指令

符号定义伪指令的用途是给表达式赋予一个符号名或为标号/变量定义新的类型属性。表达式可以是汇编语言中的变量名、标号名、过程名、寄存器名以及指令助记符等，符号可自行定义。在以后的程序中凡需要用到该表达式的地方都可用此符号名来代替。

常用的符号定义伪指令有：等值语句EQU、等号语句＝、LABEL



4.4伪指令

1. 符号定义伪指令

格式：

符号名	EQU / =	表达式
名字	LABEL	类型

▲ 作用：

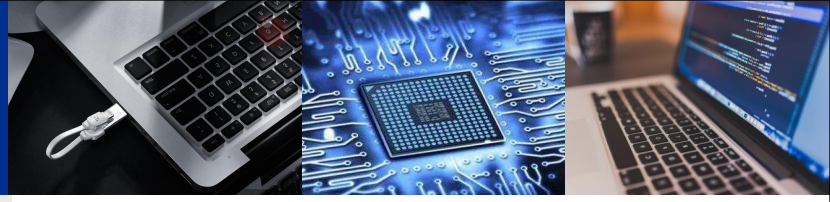
给程序中的表达式赋予一个符号名，其中表达式可以是任何有效的操作数。

汇编时用语句中的表达式代替程序中符号所在的地方。

同一符号不能赋几个不同的值。

▲ 应用：

1. 定义符号常量，方便修改程序。
2. 某表达式多次出现时，用等值伪操作可以方便编程。



4.4伪指令

可在语句中进行运算，如：

```
DATA_PORT   EQU  3F8H
STAT_PORT   EQU  3F8H+2
SEED        EQU  10
FUNC        EQU  SEED*SEED+2*SEED+1
```

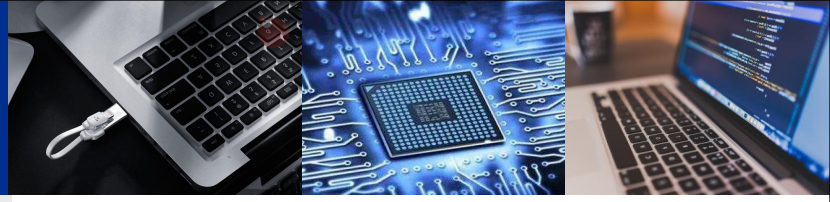
‘=’ 号操作

对符号的赋值还可使用“=”操作，它与EQU的区别是“=”操作可以重复定义，而EQU则不能重复定义。如：

```
X=3
Y=6
Y=Y*Y-X
.....
MOV  AX, Y
```

通过上述操作，最后使Y=33，后面的MOV语句中，当生成目标代码时将用33取代Y。

即 MOV AX, 21H (33D)



4.4伪指令

2.数据定义伪指令

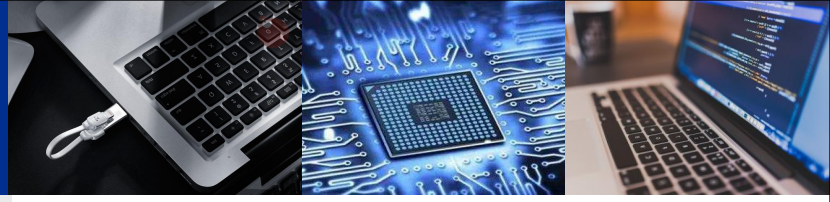
格式:

[名称] 伪指令助记符 数据表 ; [注释]

其中变量名用符号地址表示，后面不能跟冒号。

伪指令助记符有如下几种:

DB	用来定义字节	DW	用来定义字
DD	用来定义双字	DF	用来定义6字节内容
DQ	用来定义4字内容	DT	用来定义10字节内容



4.4伪指令

3.段定义伪指令

段定义伪指令的用途是在汇编语言程序中定义逻辑段，用它来指定段的名称和范围，并指明段的定位类型、组合类型及类别。

常用的段定义伪指令有 **SEGMENT / ENDS** 和 **ASSUME**等。

（1）段定义伪指令　　**SEGMENT/ENDS**

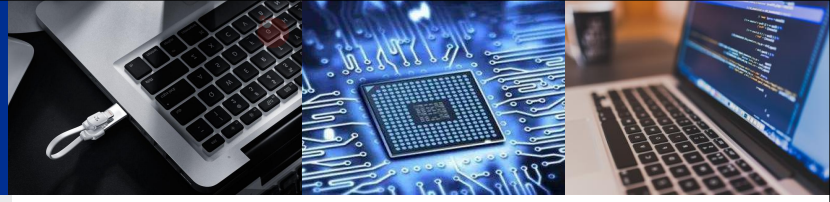
格式 ——

段名 **SEGMENT**［定位类型］［组合类型］［分类名］

段体　　；本段程序内容(指令语句或伪指令语句)

段名　**ENDS**

段名为所定义段的名称或标识符，用于识别段，指示汇编程序为该段分配的存储器起始位置。



4.4伪指令

段名 SEGMENT［定位类型］［组合类型］［分类名］

段体 ；本段程序内容(指令语句或伪指令语句)

段名 ENDS

▲ 段定义由伪指令SEGMENT开始、ENDS结束。

其中： SEGMENT 和ENDS 必须成对出现，

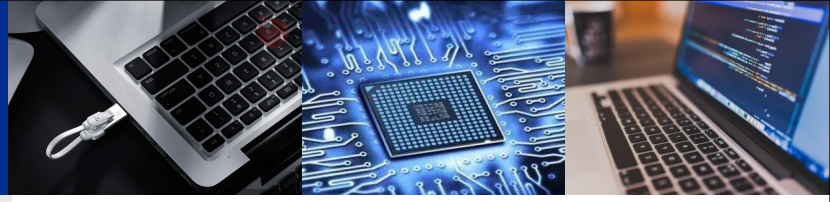
且语句前必须有段名，段名必须相同。

▲ SEGMENT和ENDS语句之间可以有指令和其他伪操作，

表示存放在该段内存的变量、指令或其他伪操作对该段内存的处理。

▲ 程序中可以定义多个段，如数据段、代码段、堆栈段、附加段。

▲ 程序经汇编、链接及装入内存后，段名为一具体的段值。



4.4伪指令

① 定位方式（定位类型）　（Align）

说明如何确定逻辑段的边界。定位类型有四种：

1）PARA（Paragraph）(缺省情况)

起始地址　××××　××××　××××　××××　0000

2）BYTE

起始地址　××××　××××　××××　××××　××××

3）WORD

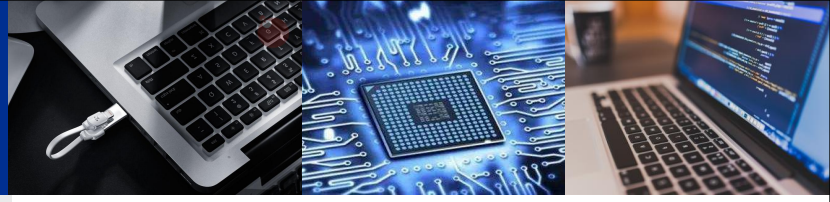
起始地址　××××　××××　××××　××××　×××0

4）DWORD

起始地址　××××　××××　××××　××××　××00

4）PAGE

起始地址　××××　××××　××××　0000 0000



4.4伪指令

② 组合类型：

一个汇编语言源程序往往有许多模块组成，而每一个模块常常又有自己的数据段、代码段和堆栈段。组合类型用于告诉LINK程序，多个模块连接是本段与其他模块中同名段的组合连接关系。

NONE：无组合关系。（默认）为默认值，表示本段和其他段不发生关系，具有独立的段地址

PUBLIC：依次连接。将同名同类型的段相邻地连接到一起，具有共同的段地址。

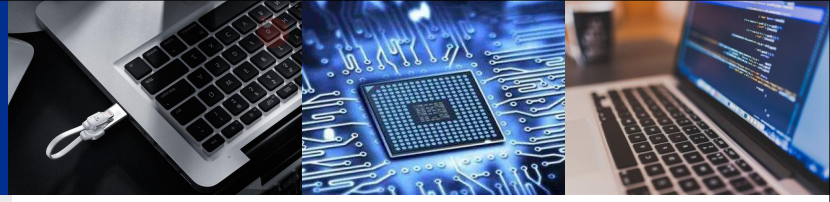
STACK：与PUBLIC方式相同，但仅用于堆栈段。

COMMON：所有同名同类型段具有相同段地址，但后面的会覆盖前面的，用于共享数据。

MEMORY：装在被链接的其它段之后。

AT表达式：相应段定位在表达式的值指定的段地址上。

③ 类别名：由单引号括起来的字符串，表示该段的类别，如’STACK’、‘CODE’、’DATA’等。



4.4伪指令

(2) 段寄存器说明伪指令　　ASSUME

格式：ASSUME　段寄存器：　段名　[，段寄存器:段名，…]

其中：段寄存器为CS、DS、ES、SS中的一个，

段名为用SEGMENT段定义伪指令定义过的段名。

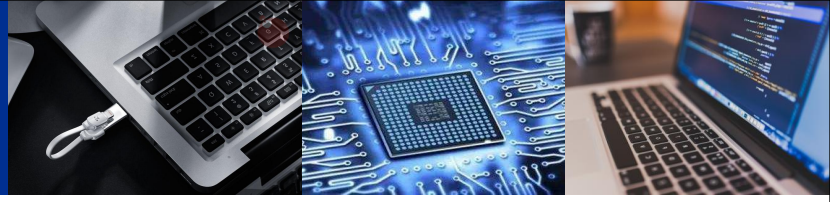
例　　ASSUME　CS: cc ，DS:aa

▲　ASSUME 伪操作的作用

设定特定的段寄存器指向特定的段，说明源程序中定义的段由哪个段寄存器去寻址。不如此，汇编程序无法生成目标代码程序。

▲ ASSUME语句只起指示作用，并无实际的操作。

ASSUME并未真正将段地址装入相应的段寄存器，段寄存器(CS除外)的初值设定还是要由程序中的MOV指令来完成。



程序中有ASSUME语句，经汇编，链接和装入内存后的情况：

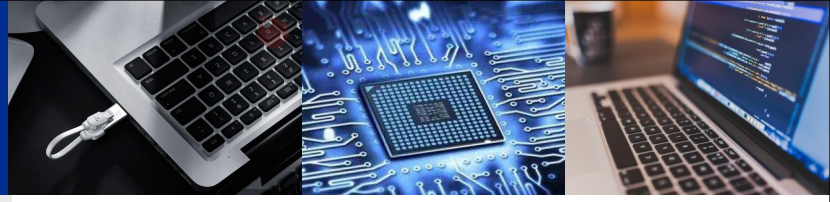
```
data    SEGMENT
value   DW    1234H, 5678H
result  DW    ?
data    ENDS
code    SEGMENT
        ASSUME  CS:code, DS:data
start:  MOV   AX, data      ;给DS赋值
        MOV   DS, AX
        MOV   AX, value     ;取数
        ADD   AX, value+2   ;两数相加
        MOV   result, AX    ;保存结果
        MOV   AH, 4CH       ;返回DOS
        INT   21H
code    ENDS
        END start
```

```
D:\MASM>DEBUG  assume.exe
-R              ;查看程序执行前各寄存器
AX=0000  BX=0000  CX=0023  DX=0000
SP=0000  BP=0000  SI=0000  DI=0000
DS=1295  ES=1295  SS=12A5
CS=12A6  IP=0000
NV UP EI PL NZ NA PO NC
12A6:0000 B8A512      MOV     AX,12A5
-U              ;查看在内存的程序
12A6:0000 B8A512      MOV     AX, 12A5
12A6:0003 8ED8        MOV     DS, AX
12A6:0005 A10000      MOV     AX, [ 0000 ]
12A6:0008 03060200    ADD     AX, [ 0002 ]
12A6:000C A30400      MOV     [ 0004 ], AX
12A6:000F B44C        MOV     AH, 4C
12A6:0011 CD21        INT     21
……
-
```

注意：

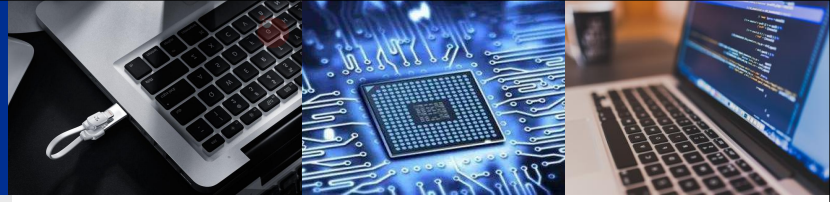
程序装入内存后，执行程序前，

当前的DS值并非程序定义的data段值， 1295 不等于 12A5。



宏指令

- 为减少重复编写相同语句段的工作，我们可以使用“宏”功能
- 宏定义是指用一个标识符（宏名）来代替一组指令序列
- 宏功能的使用过程是：宏定义、宏调用、宏展开。



宏指令

1. 宏定义

宏指令定义格式：

```
宏指令名  MACRO  <形参列表>
        汇编程序段(宏体)
ENDM
```

MACRO 与 ENDM必须成对出现，先定义后引用

宏名

```
SHIFT  MACRO
    MOV  CL,  4
    SAL  AL, CL
ENDM
```

程序段: 将AL左移4位(乘16)

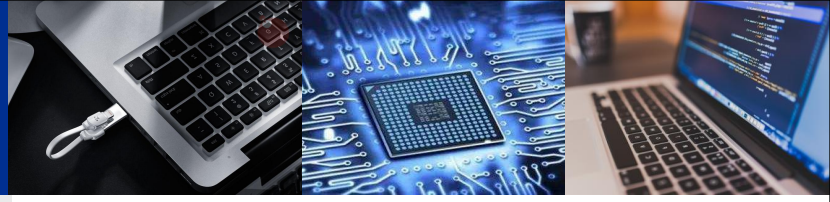
2. 宏调用

宏指令调用格式：

```
宏指令名  [实参列表]
```

在程序中引用宏指令如下：

```
IN  AL,  5FH
SHIFT
OUT  5FH,  AL
```



宏指令

宏指令也可以接收参数，如对X左移Y位。

```
SHIFT  MACRO   X, Y
    MOV  CL,  Y
    SAL  X, CL
ENDM
```

形参列表

BX→X

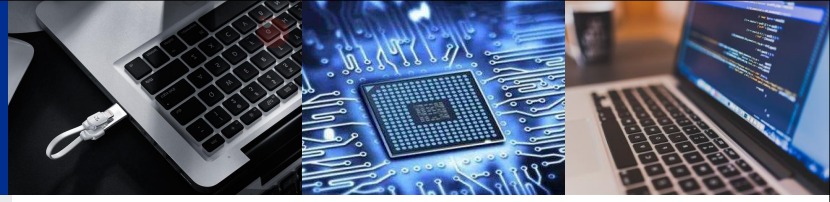
4→Y

实参形参一一对应

在程序中引用宏指令如下：

```
MOV  BX,  WORD PTR  BUF
SHIFT   BX,  4
MOV  WORD PTR BUF ,  BX
```

功能：BX的值左移4位。



宏指令

3. 宏展开

源程序在汇编时，
过程称为宏展开。

定义键盘输入宏指

```
INPUT
ENDM
```

定义实现源程序结

```
RETSYS
ENDM
```

展开前：

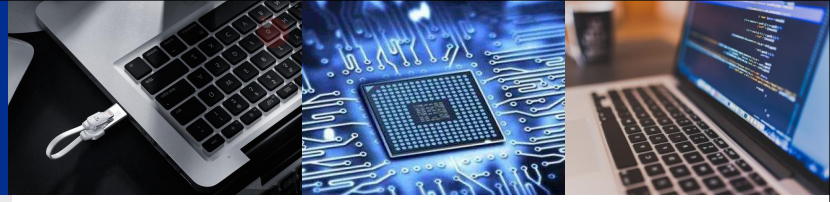
```
.MODEL SMALL
.STACK 100H
.CODE
START:
INPUT

CMP  AL, '-'
JNE  START
RETSYS

END  START
```

展开后：

```
START:
+       MOV  AH，01H
+       INT  21H
        CMP  AL, '-'
        JNE  START
+       MOV  AH，4CH
+       INT  21H
```

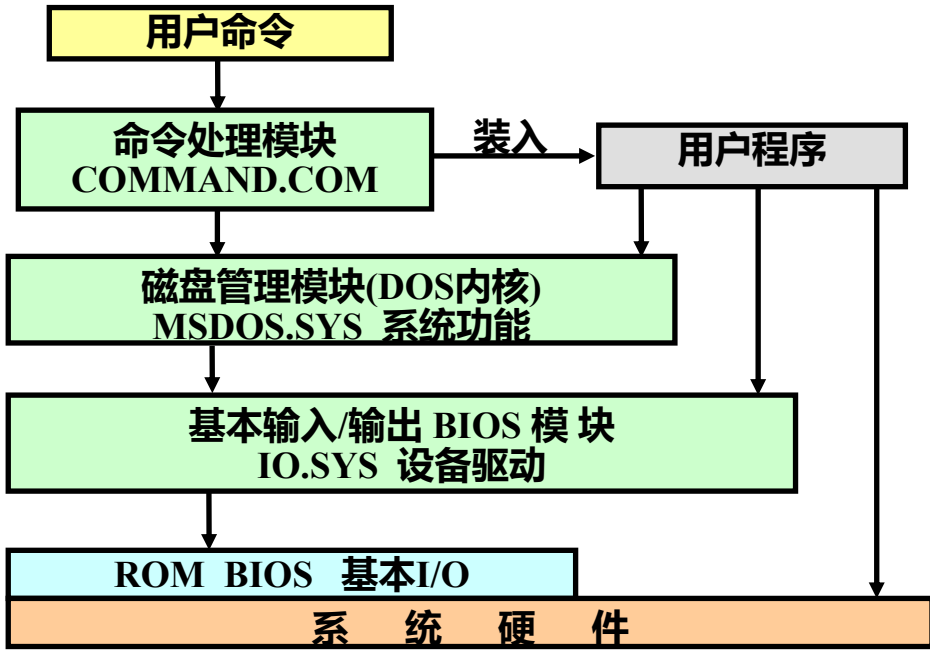


4.5系统功能调用

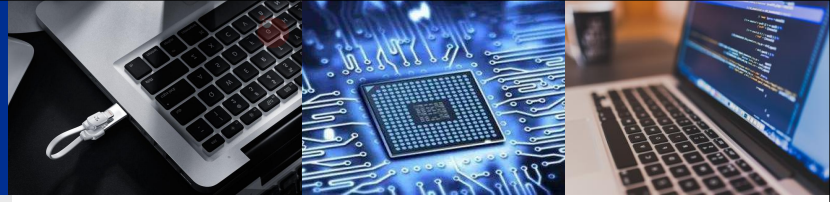
为节省编程工作量与优化程序结构，在DOS及BIOS中预先设计好了一系列通用子程序，以便供DOS及BIOS调用。

由于这种调用采用的是以中断指令INT n的内部中断方式进行的，所以常称为DOS及BIOS中断调用。

在一个中断服务程序中往往包含有多个功能相对独立的子程序，所以也将中断调用称为系统功能调用或功能调用或中断功能调用。



DOS系统层次结构



4.5系统功能调用

操作系统提供了许多通用子程序供程序员使用,这些子程序分别用于I/O管理、内存管理、文件管理、作业管理。

4.4.1 系统功能调用方法

格式:

INT n

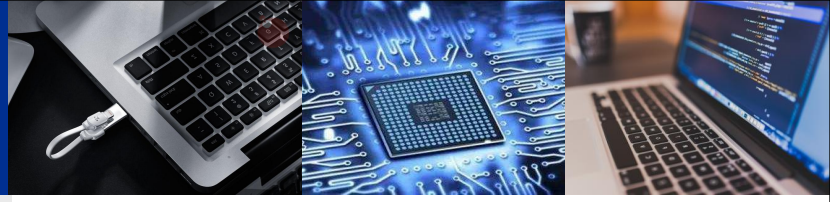
其中，n是中断类型码。当n=4～1FH时，调用BIOS中的服务程序，称作系统中断调用；当n=20～3FH时，调用DOS中的服务程序，称作功能调用。

调用的方法如下:

① AH或AX←功能号;

② 在其它寄存器中放入该功能所要求的入口参数;

③ 执行INT n指令;

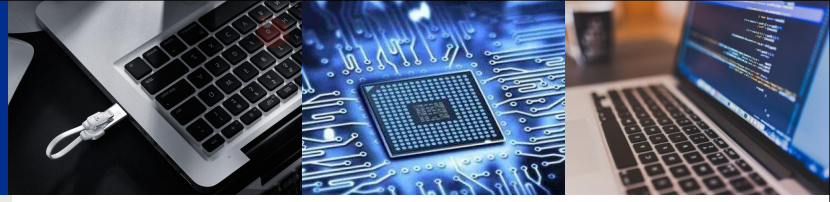


4.5系统功能调用

4.5.2 BIOS调用

- BIOS常驻ROM，独立于DOS，可与任何操作系统一起工作。
- 它的主要功能是驱动系统所配置的外部设备，如磁盘驱动器、显示器、打印机及异步通信接口等。
- 通过INT 10H～INT 1AH向用户提供服务程序的入口，使用户无须对硬件有深入了解，就可完成对I/O设备的控制与操作。
- BIOS的中断调用与DOS功能调用类似。

软中断指令	功 能	软中断指令	功 能
INT 00H	除法出错	INT 0DH	硬盘中断
INT 01H	单步中断	INT 0EH	软盘中断
INT 02H	非屏蔽中断	INT 10H	显示器中断
INT 03H	断点中断	INT 12H	内存大小检查
INT 04H	溢出中断	INT 15H	盒式磁带机I/O
INT 09H	键盘中断	INT 16H	键盘输入
INT 0BH	异步通信串行口1中断	INT 17H	打印机输出
INT 0CH	异步通信串行口2中断	INT 1AH	时钟



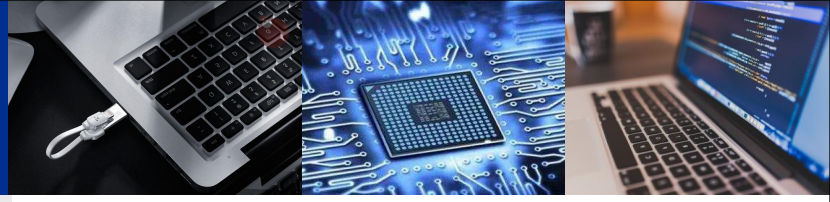
4.5系统功能调用

BIOS与DOS功能调用比较：

1. 调用BIOS功能子程序比调用DOS功能子程序要复杂一些，但运行速度快，功能更强。

2. DOS功能调用只是在DOS的环境下使用，而BIOS功能调用不受任何操作系统的约束。

3.某些功能只有BIOS具有。



4.5系统功能调用

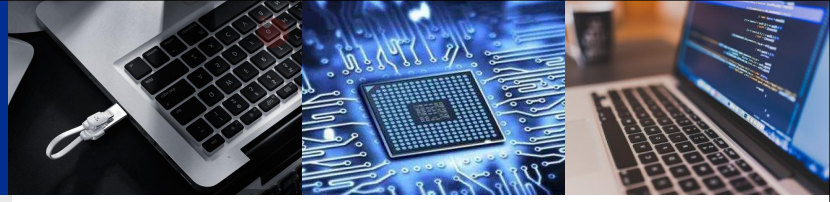
4.5.3 DOS系统功能调用

DOS功能调用：指DOS为系统程序员和用户提供的一组常用子程序。

用中断指令INT 21H进入各功能调用子程序的总入口，再为每个功能调用规定一个功能号以便进入相应各子程序的入口。

子程序的入口参数及出口参数在每个功能调用的说明中可以查到。

在DOS中断服务程序中，有近百个功能供用户选择使用，其中，功能最多的是向量号为21H的向量中断。

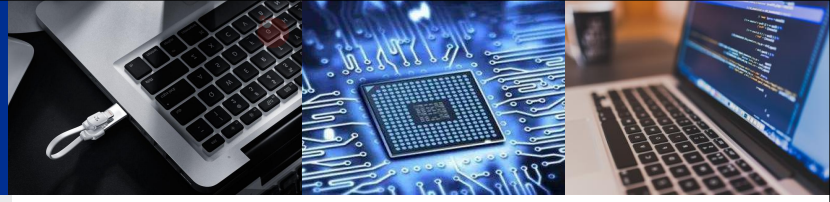


4.5系统功能调用

1. 键盘输入并显示(01号功能调用)

- 功能调用号AH=01H。
- 功能：接收从键盘输入的一个字符并在屏幕回显。输入字符的ASCII码存入AL寄存器。若按下组合键Ctrl+Break或Ctrl+C，则程序返回DOS；如果按下的是Tab键，屏幕上光标自动移至下一个制表位。
  - 入口参数：无
  - 出口参数：AL寄存器存放输入字符的ASCII码值
- 格式：

```
MOV  AH,01H
INT  21H
```

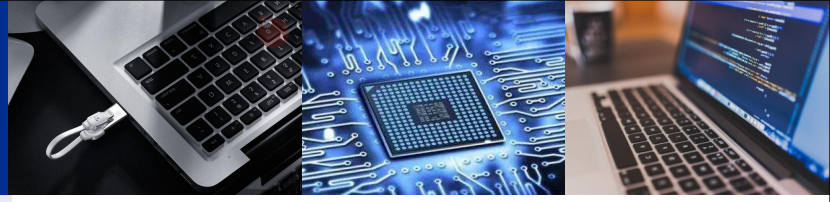


4.5系统功能调用

2.输出单字符(02H号功能)

- 功能调用号AH=02H。
- 功能：在屏幕上显示一个字符。
  - 入口参数：DL=输出字符
  - 出口参数：无
- 例如：

```
MOV DL，'A'                    ；A字符的ASCII码置入DL中
MOV AH，2
INT 21H
```

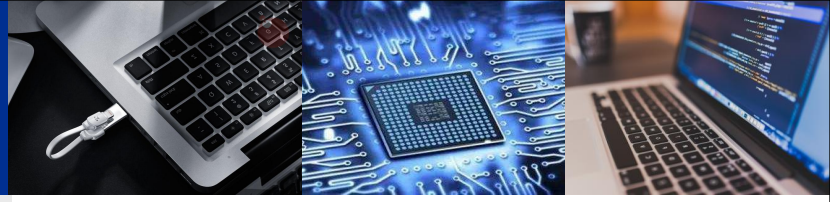


4.5系统功能调用

3.不带显示的单字符键盘输入(07H号、08H号功能)

- 功能调用号AH=07H(08H)。
- 功能：与01H号功能类似，区别仅仅是输入的字符不在屏幕上显示。其中，07H号功能调用对Ctrl+C键和Tab键无反应。
  - 入口参数：无
  - 出口参数：AL=输出字符
- 格式：

```
MOV    AH , 07H
INT 21H
或
MOV    AH，08H
INT 21H
```



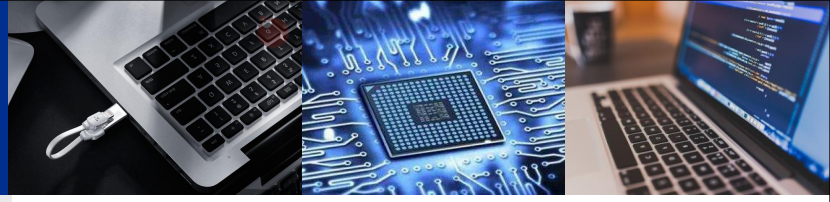
4.4系统功能调用

4.字符串输出(09H号功能)

- 功能调用号AH=9。
- 功能：在屏幕上显示一个字符串。
  - 入口参数：是被输出字符串首址，接收入口参数的是寄存器DS和DX，分别存入被输出字符串首址的段基值和偏移量。
    - 采用9号功能输出字符串，要求字符串以‘$’结束，该字符作为字符串结束符，不输出。
  - 出口参数：无

格式：

```
MOV     DX，字符串偏移量
MOV     AH，09H
INT     21H
```



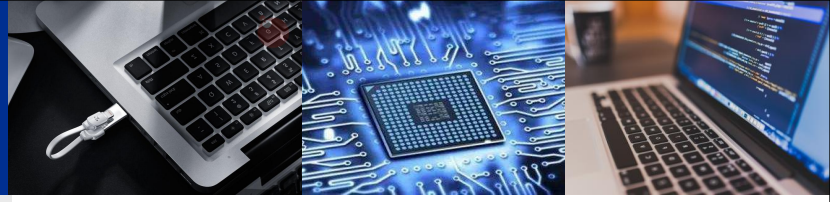
4.4系统功能调用

字符串输出（续）

例如：

```
DATA  SEGMENTS
      STRING     DB       'A EXAMPLE'0DH’，0AH，'$’;定义字符串
      …
DATA  ENDS

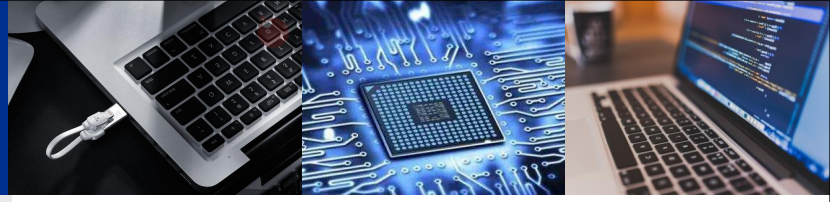
CODE  SEGMENT
      …
      MOV  DX,OFFSET  STRING
      MOV  AH,9
      INT  21H
      …
CODE  ENDS
```



4.4系统功能调用

4.字符串输入(0AH号功能调用)

- 功能调用号AH=0AH。
- 功能：从键盘接受字符串后存入以DS:DX为首地址的内存缓冲区，同时显示该字符串，输入过程以回车键结束。要求先建立缓冲区，并且：
  - ✓ 1. 第一个字节缓冲区能容纳的字符个数
  - ✓ 2. 第二个字节保留，以存放实际放入的字符个数
  - ✓ 3. 第三个字符开始存放从键盘输入的字符。
  - ✓ 4. 实际输入不足，则补0；太多，则忽略。
  - ➢ 入口参数: DS: DX=缓冲区首址
    [DS: DX]=缓冲区最大字符个数
  - ➢ 出口参数: [DS: DX+1] =实际输入的字符个数
    [DS: DX+2]单元开始存放实际输入的字符



4.4系统功能调用

字符串输入(续)

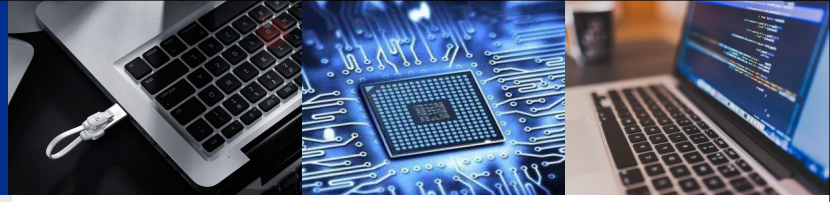
格式:

```
MOV    DX，缓冲区偏移量
MOV    DS，缓冲区段基址
MOV    AH，0AH
INT    21H
```

例如:

```
BUF    DB 30，？，30 DUP(？)

MOV    DX，OFFSET BUF
;MOV   DS，SEG  BUF
MOV    AH，0AH
INT    21H
```



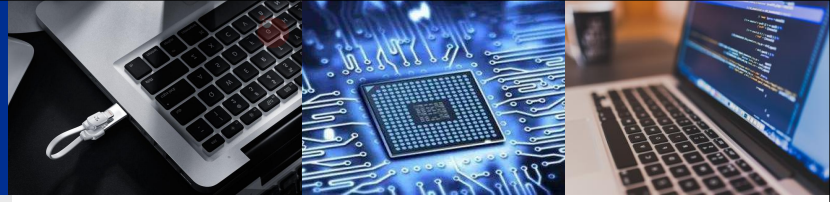
4.4系统功能调用

6. 返回操作系统(4CH号功能)

- 功能调用号AH=4CH。
- 功能：使系统结束程序运行后返回DOS状态。。
  - 入口参数： AL=返回码
  - 出口参数：无

格式:

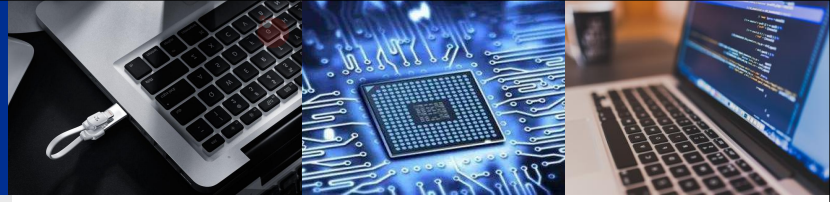
```
MOV     AH，4CH
INT     21H
```



4.7汇编语言程序设计举例

汇编源程序的设计步骤

- 分析问题，建立数学模型
- 确定最佳算法
- 合理分配存储单元和寄存器
- 绘制流程图
- 编写程序
- 调试程序

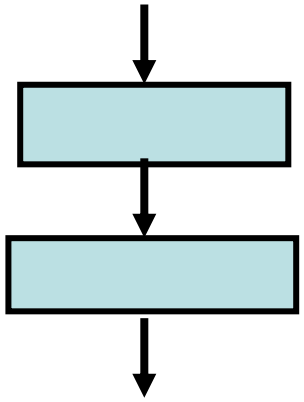


4.7汇编语言程序设计举例

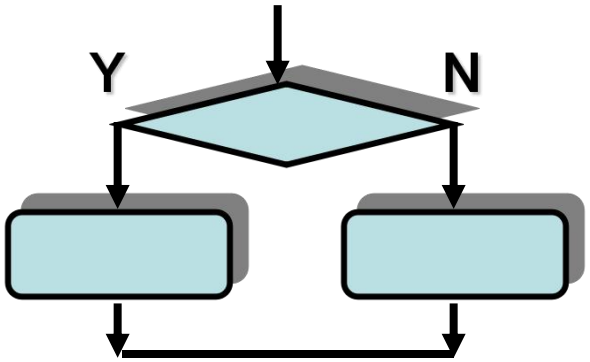
4.7.1 程序基本结构

- 模块化设计方法
  - ✓ “自顶向下，逐步细化”
- 结构化编码方法
  - ✓ 顺序、分支、循环三种基本结构

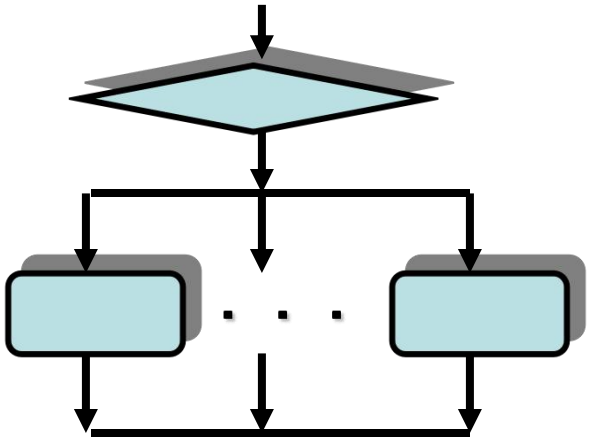
顺序程序结构

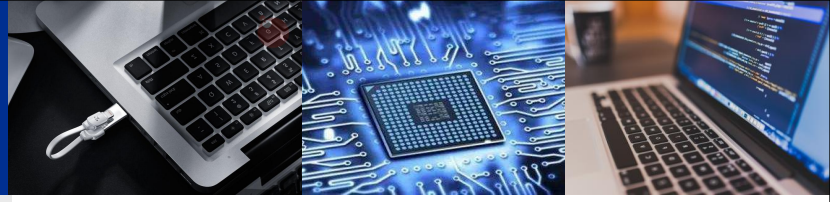


条件程序结构



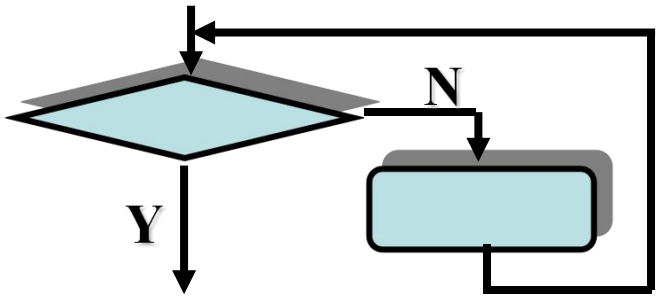
多分支程序结构



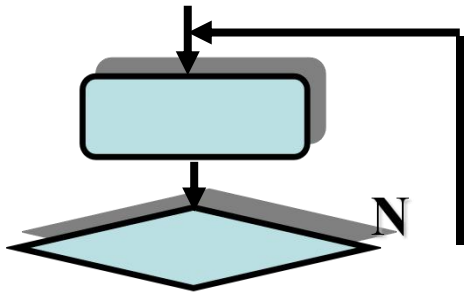


4.7汇编语言程序设计举例

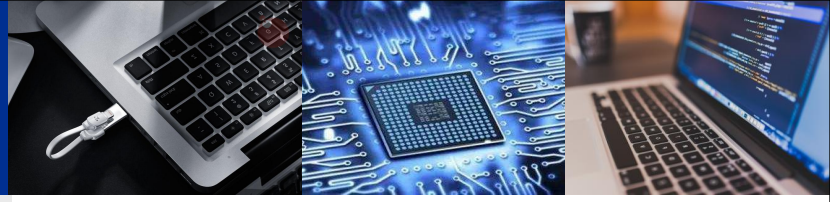
循环控制结构 (当型循环)



循环控制结构 (直到型循环)



- **顺序程序设计** 指令指针 IP 值线性增加，IP = IP + 1
- **条件程序设计** IP 值受标志位的影响而跳变
  影响标志的指令 CMP、TEST、JXX
- **循环程序设计** IP 值受计数器 CX 中的值不为零而循环
  影响标志的指令 DEC



4.7汇编语言程序设计举例

4.7.2顺序结构程序设计

例4.6

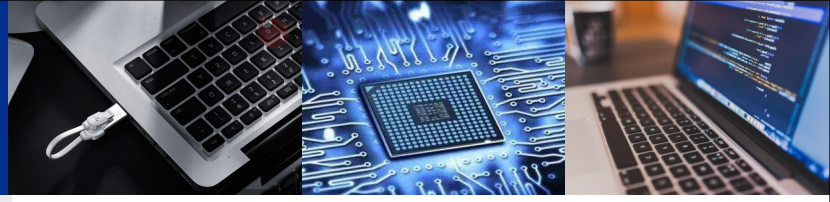
编程序计算： SUM=3*(X+Y)+(Y+Z)/(Y-Z)，其中，X、Y、Z都是16位无符号数，要求结果存入SUM单元。假设运算过程中，中间结果都不超出16位二进制数的范围。

设计思路：

（1）在数据段中定义3个字型变量X、Y、Z 。

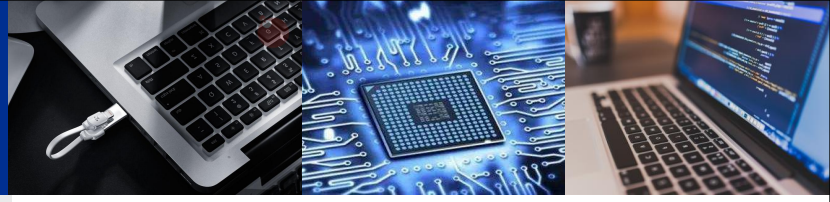
（2）采用带符号数乘除指令，要注意操作数的属性问题。

（3）假定X、Y、Z的值如程序所示，则(Y+Z)/(Y-Z)结果应为：商AX中。



4.7汇编语言程序设计举例

```
DATA    SEGMENT
        X   DW  6
        Y   DW  8
        Z   DW  7
      SUM   DW  ?
DATA    ENDS
CODE    SEGMENT
        ASSUME  CS:CODE, DS:DATA
START:  MOV  AX, DATA
        MOV  DS, AX
        MOV     AX，X       ; 取X
        ADD     AX，Y       ; AX←X+Y，乘法操作数1
        MOV     CX，3       ；乘法操作数2
        MUL     CX          ; DX: AX←3*(X+Y)
        MOV     CX，AX      ; CX←3*(X+Y)保存
        MOV     AX，Y       ; 取Y
        ADD     AX，Z       ; AX←Y+Z，被除数
        XOR     DX，DX      ; DX←0
        MOV     BX，Y       ; 取Y
        SUB     BX，Z       ; BX←Y-Z，除数
        DIV     BX          ; AX←(Y+Z)/(Y-Z)的商
        ADD     AX，CX      ; AX←3*(X+Y)+(Y+Z)/(Y-Z)，两项之和
        MOV     SUM，AX     ; 存结果
        MOV  AH, 4CH
        INT  21H
CODE    ENDS
        END START
```

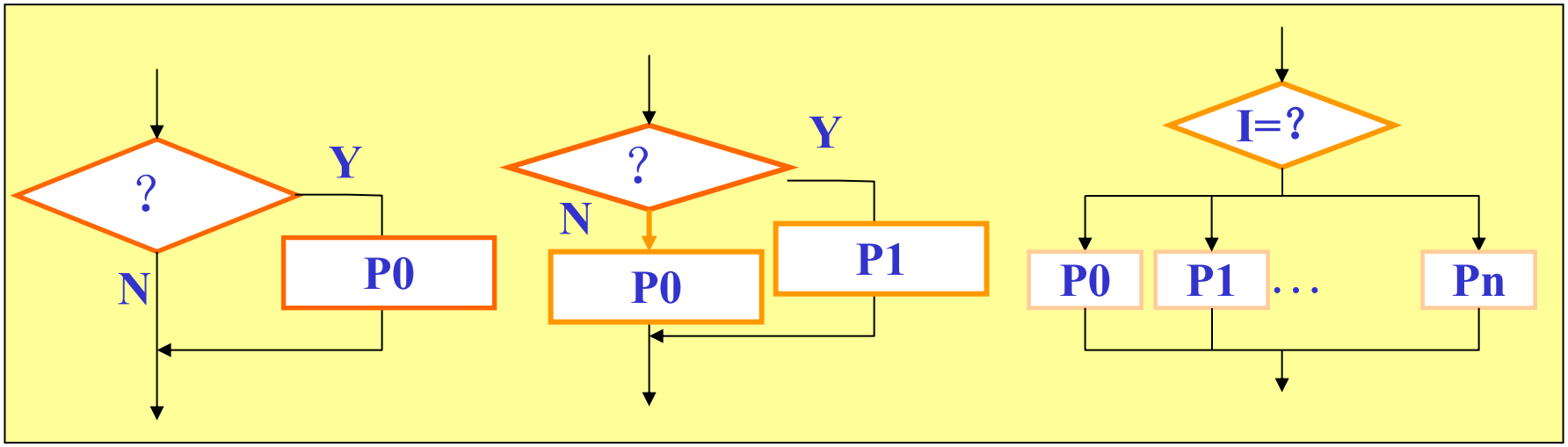


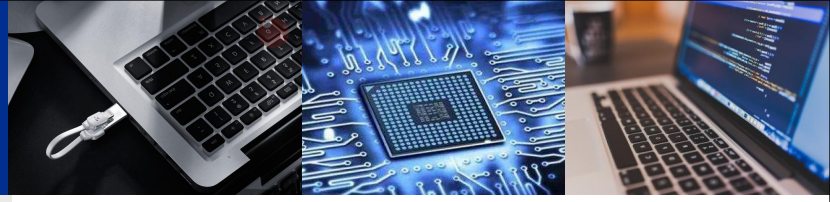
4.7汇编语言程序设计举例

4.7.3分支结构程序设计

1.分支程序的结构形式

单纯分支、 并行分支结构、选择分支结构

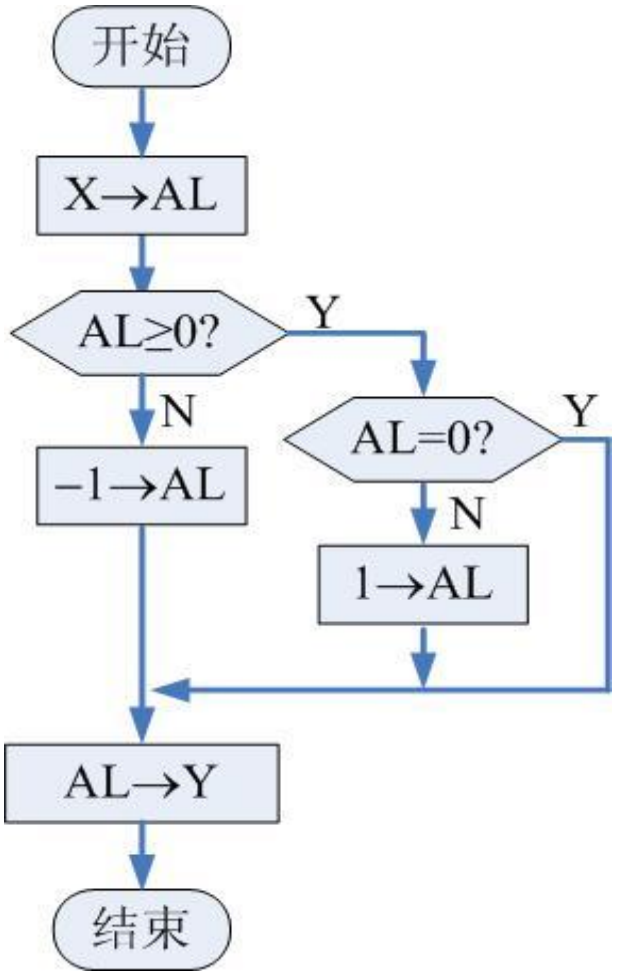




4.7汇编语言程序设计举例

例4.8：符号函数

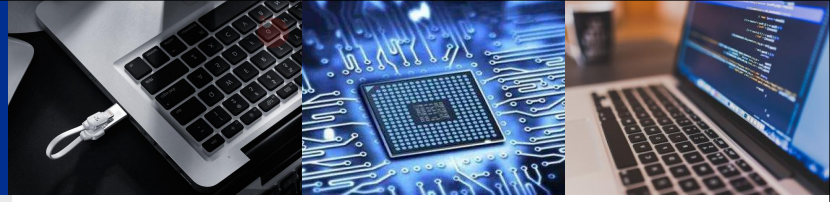
$$Y=\begin{cases}1 & X>0\\ 0 & X=0\\ -1 & X<0\end{cases}$$



◆ 设计思路：

1.设X的范围是[-128,127],取出X放至AL寄存器

2. 比较AL里面的数和0的关系，转至3个分支



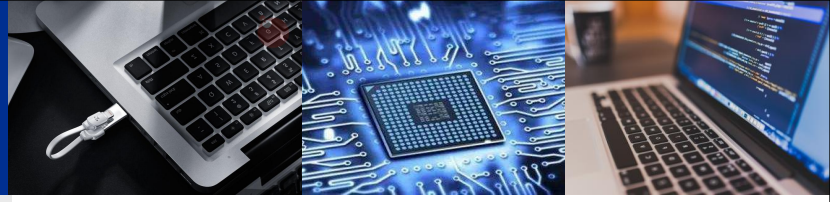
4.7汇编语言程序设计举例

```
DATA   SEGMENT
   X  DB  -18
   Y  DB  ?
DATA   ENDS

CODE   SEGMENT
       ASSUME CS:CODE,
                    DS:DATA

START:    MOV AX, DATA
          MOV DS, AX
```

```
MOV AL, X
CMP     X，0
JL      PNUM ; X＜0转移到PNUM
JZ      ZERO  ; X=0转移到ZERO
MOV     Y，1
JMP     EXIT    ; X>0
PNUM:   MOV   Y，-1
JMP     EXIT    ; X<0
ZERO:   MOV   Y，0; X=0
EXIT:   MOV Y, AL
        MOV AH, 4CH
        INT 21H
CODE    ENDS
        END START
```



4.7汇编语言程序设计举例

跳跃表法

多分支结构的实现，还可以使用跳跃表法，使程序能根据不同的条件转移到多个程序分支中去。

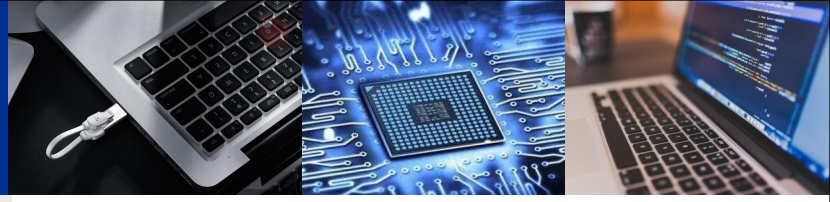
需要在数据段事先安排一个按顺序排列的转移地址表。

输入的数字作为偏移量。因为只有2个字节16位偏移地址，所以偏移量需要乘2。

关键是要理解间接寻址方式JMP指令

```
Table    dw   disp1, disp2,      disp3, disp4, ...
```



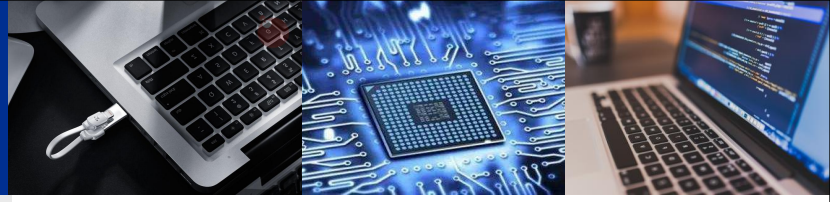


4.7汇编语言程序设计举例

例4.9 变址寻址方式

根据AL寄存器中哪一位为1（从低位到高位）把程序转移到8个不同的程序分支中去。

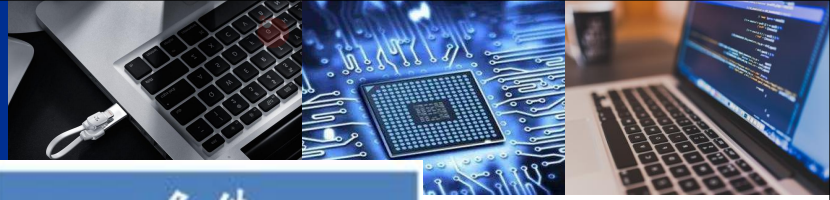
```
DATA      SEGMENT
DATATAB           DW  ROUTINE_1
                  DW  ROUTINE_2
                  DW  ROUTINE_3
                  DW  ROUTINE_4
                  DW  ROUTINE_4
                  DW  ROUTINE_6
                  DW  ROUTINE_7
                  DW  ROUTINE_8
DATA      ENDS
```



4.7汇编语言程序设计

```
DATA    SEGMENT
DATATAB DW  ROUTINE_1
        DW  ROUTINE_2
        DW  ROUTINE_3
        DW  ROUTINE_4
        DW  ROUTINE_4
        DW  ROUTINE_6
        DW  ROUTINE_7
        DW  ROUTINE_8
DATA    ENDS
```

```
CODE    SEGMENT
MAIN    PROCFAR
        ASSUMECS: CODE，DS: DATA
START:  PUSH   DS
        SUB    BX，BX
        PUSH   BX
        MOV    BX，DATA
        MOV    DS，BX
        CMP    AL，0
        JE     CONT
        MOV    SI，0
LP:     SHR    AL，1
        JNB    NOT_YET ;CF=0
        JMP    DATATAB［SI］
NOT_YET: ADD   SI，TYPE  DATATAB
        JMP    LP
CONT:   （不发生转移）
……
ROUTINE_1:          ……
ROUTINE_2:          ……
……
        RET
MAIN    ENDP
CODE    ENDS
END             START
```



指令	条件	指令	条件
ja	CF=0 and ZF=0	jnc	CF=0
jab	CF=0	jne	ZF=0
jb	CF=1	jng	ZF=1 or SF!=OF
jbe	CF=1 or ZF=1	jnge	SF!=OF
jc	CF=1	jnl	SF=OF
jcxz	CX=0	jnle	ZF=0 and SF=OF
je	ZF=1	jno	OF=0
jecxz	ECX=0	jnp	PF=0
jg	ZF=0 and SF=OF	jns	SF=0
jge	SF=OF	jnz	ZF=0
jl	SF!=OF	jo	OF=1
jle	ZF=1 or SF!=OF	jp	PF=1
jmp	无条件跳转	jpe	PF=1
jna	CF=1 or ZF=1	jpo	PF=0
jnae	CF=1	js	SF=1
jnb	CF=0	jz	ZF=1
jnbe	CF=0 and ZF=0		

鱼C工作室 FISHC.COM